

会議室予約システム 設計・コード解説 第2版

C# / ASP.NET MVC 学習3～4か月向け

対象	現在のリポジトリ実装
構成	本文 + Controller / Service / DAOメソッド解説
技術	ASP.NET MVC 5 / .NET Framework 4.7.2 / SQLite
DBアクセス	Entity Framework不使用 / DAO / 生SQL / SQLiteParameter

Controller -> Service -> DAO -> SQLite
の順に処理を追えるよう、初心者向けの言葉で説明しています。

この資料について

この資料は、C#とASP.NET MVCを学び始めて3～4か月ほどの人を対象にしています。

目標は、現在のコードを読んだときに次のことを自分の言葉で説明できるようになることです。

- どのクラスが何を担当しているか
- 画面からDBまで、処理がどの順番で進むか
- 各メソッドが何を確認し、何を行い、何を返すか
- なぜService、DAO、Entity、ViewModelを分けているか

この第2版は、現在のリポジトリに存在するコードを基準にしています。

メソッドの詳しい説明は、読みやすいように次の3ファイルへ分けています。

- Controllerメソッド一覧
- Serviceメソッド一覧
- DAOメソッド一覧

1. システム全体概要

このシステムは、社内の会議室を検索し、本予約・仮予約・変更・キャンセルを行うWebアプリケーションです。

使用している主な技術は次のとおりです。

Webフレームワーク	ASP.NET MVC 5
実行環境	.NET Framework 4.7.2
View	Razor 3
DB	SQLite
DBアクセス	DAOパターンと生SQL
SQLの値渡し	SQLiteParameter
Entity Framework	使用していない

現在はログイン機能が未実装です。Controllerでは、ログイン中のユーザーを一時的に**U001**として扱います。

RouteConfigの既定ルートは**ReservationListController.Index**です。そのため、アプリのルートURLを開くとホームの予約一覧が表示されます。

予約状態

0	Tentative	仮予約
1	Confirmed	本予約
2	Cancelled	キャンセル済み

「実施済み」というenumはありません。次の条件で画面表示時に判断します。

本予約 = Confirmed かつ EndDateTime >= 現在時刻
実施済み = Confirmed かつ EndDateTime < 現在時刻
仮予約 = Tentative

Cancelled予約はホーム画面の一覧タブには表示しません。

主な業務ルール

- ・ 営業時間は09:00～18:00
- ・ 予約は15分単位
- ・ 仮予約はユーザー1人につき3件まで
- ・ ConfirmedとTentativeは会議室の空き判定対象
- ・ Cancelledは空き判定対象外
- ・ 正式料金はServiceがDBのHourlyRateを使って計算
- ・ 予約登録・変更の直前に、空き状況をもう一度確認
- ・ 日時はJSTローカル時刻をISO-8601形式のTEXTとして保存

2. フォルダ構成

```
ReservationManagementApp/  
├─ App_Data/  
│   ├── Initialize.sql  
│   └─ reservation.db  
├─ App_Start/  
│   └─ RouteConfig.cs  
├─ Controllers/  
│   ├── ReservationController.cs  
│   └─ ReservationListController.cs  
├─ DataAccess/  
│   ├── DaoBase.cs  
│   ├── SQLiteConnectionFactory.cs  
│   ├── FacilityDao.cs  
│   ├── RoomDao.cs  
│   ├── ReservationDao.cs  
│   └─ ParticipantDao.cs  
├─ Models/  
│   ├── Entities/  
│   └─ ViewModels/  
├─ Services/  
│   ├── ReservationSearchService.cs  
│   ├── ReservationService.cs  
│   └─ ReservationListService.cs  
└─ Views/  
    ├── Reservation/  
    ├── ReservationList/  
    └─ Shared/
```

Controllers	URLから値を受け取り、Serviceを呼び、Viewを返す
Services	予約制限、料金、Transactionなどの業務処理
DataAccess	SQLを使ってSQLiteを読み書きする
Models/Entities	DBのデータをC#で表す
Models/ViewModels	各画面に必要なデータをまとめる
Views	HTMLとフォームを表示する
App_Data	SQLite DBと初期化SQLを置く

会議室予約システム クラス図 1. 全体構成



4

4. 各クラスの役割

Controller

ReservationController

新しい予約を作る画面を担当します。

- ・ 新規本予約のタイムライン検索
- ・ 新規本予約の入力・確認・登録
- ・ 新規仮予約の検索・登録

ReservationListController

すでに存在する予約を扱います。

- ・ ホーム画面と予約一覧
- ・ 仮予約から本予約への確定
- ・ 本予約の変更
- ・ 本予約と仮予約のキャンセル

Service

ReservationSearchService

会議室検索を担当します。拠点、人数、日時、仮予約上限を確認し、DAOから得た結果を画面用にまとめます。

ReservationService

DBを変更する予約処理を担当します。

- ・ 本予約作成
- ・ 仮予約作成
- ・ 仮予約から本予約への確定
- ・ 本予約変更
- ・ キャンセル
- ・ 料金計算
- ・ 営業時間と15分単位の確認
- ・ Transactionの開始、Commit、Rollback

ReservationListService

予約一覧を作ります。Reservationだけでなく、Room、Facility、Participantも読み、**ReservationDetailViewModel**へまとめます。本予約と実施済みの時間判定も担当します。

DAO

SQLiteConnectionFactory

引数なしコンストラクタが**Web.config**の接続文字列を直接読みます。設定がない場合は例外にし、**OpenConnection**で接続を開いたあと、SQLiteの外部キー制約を接続ごとに有効化します。

DaoBase

各DAOが共通で使う次の処理を持ちます。

- ・ 通常接続またはServiceから共有された接続の選択
- ・ SQLiteCommand作成
- ・ SQLiteParameter追加
- ・ JST日時とDB文字列の相互変換

FacilityDao / RoomDao / ReservationDao / ParticipantDao

それぞれ担当するテーブルをSQLで読み書きします。業務ルールや画面遷移は判断しません。

UserDaoは現在存在しません。ログイン機能が未実装で、現在の機能から使われていなかったため削除されています。将来用の**User** Entity、**UserRole** enum、**users**テーブルは残っています。

Entity

User	利用者
Facility	拠点
Room	会議室
Reservation	予約と参加者一覧
Participant	予約に入力された参加者
ReservationStatus	Tentative、Confirmed、Cancelled
UserRole	Sales、GeneralAffairs

ViewModel

ReservationScheduleViewModel	本予約のタイムライン検索結果
RoomScheduleViewModel	タイムラインの会議室1行
ReservationInputViewModel	新規予約、仮予約確定、予約変更の入力・確認
ReservationDetailViewModel	ホームの予約一覧とページ内詳細

5. 全メソッドの説明

メソッド数が多いため、層ごとに別ファイルへ分けています。

1. Controllerメソッド

2. Serviceメソッド

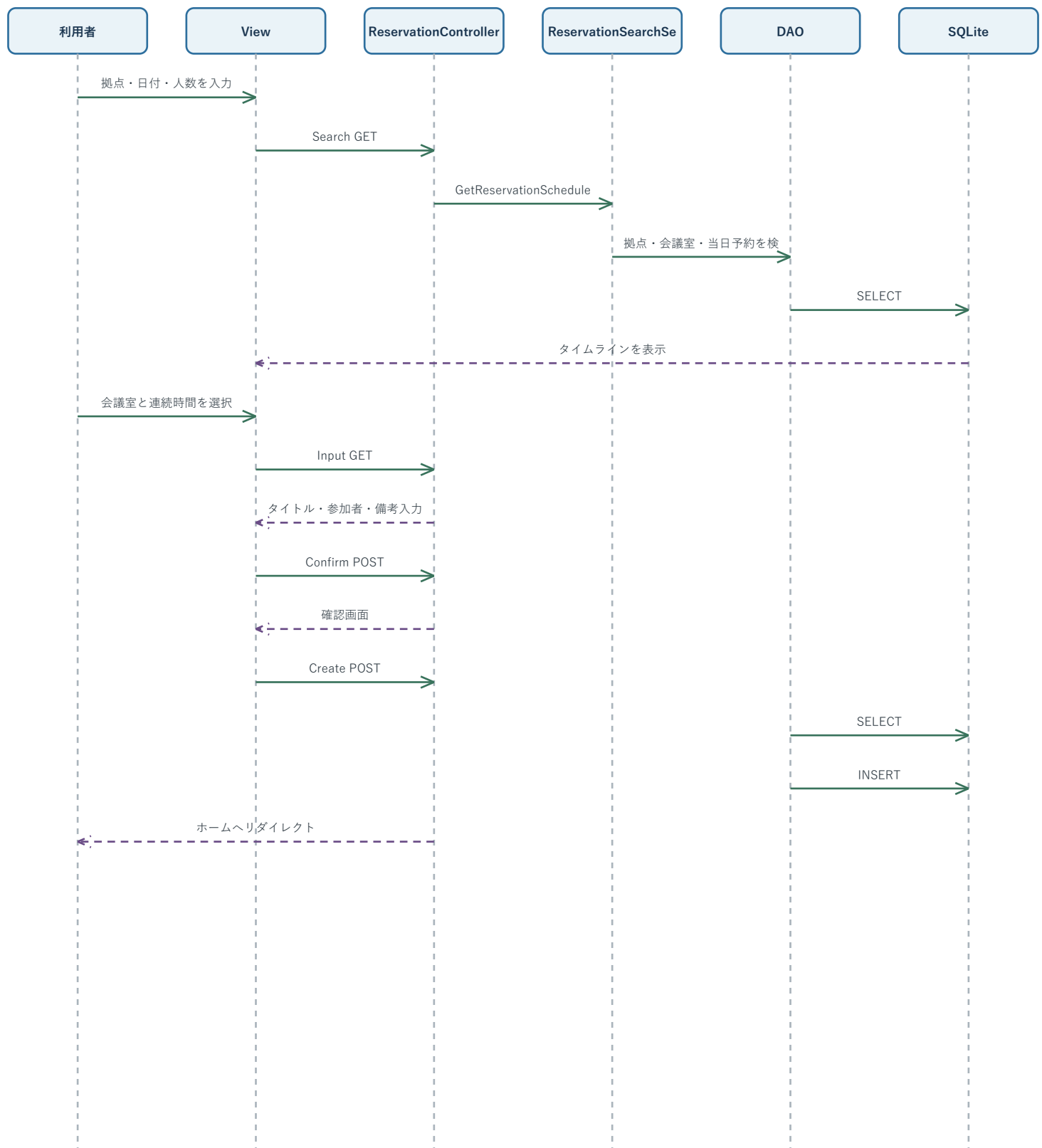
3. DAOメソッド

各ファイルには、次の情報を記載しています。

- ・ メソッド名
- ・ 引数
- ・ 戻り値
- ・ 呼び出し元
- ・ 次に呼ぶもの
- ・ 何を確認するか
- ・ 何を行うか
- ・ 最後に何を返すか

ReservationServiceの4つのTransaction対象メソッドは、Serviceメソッド一覧で処理順を細かく説明しています。

6. 新規本予約フロー



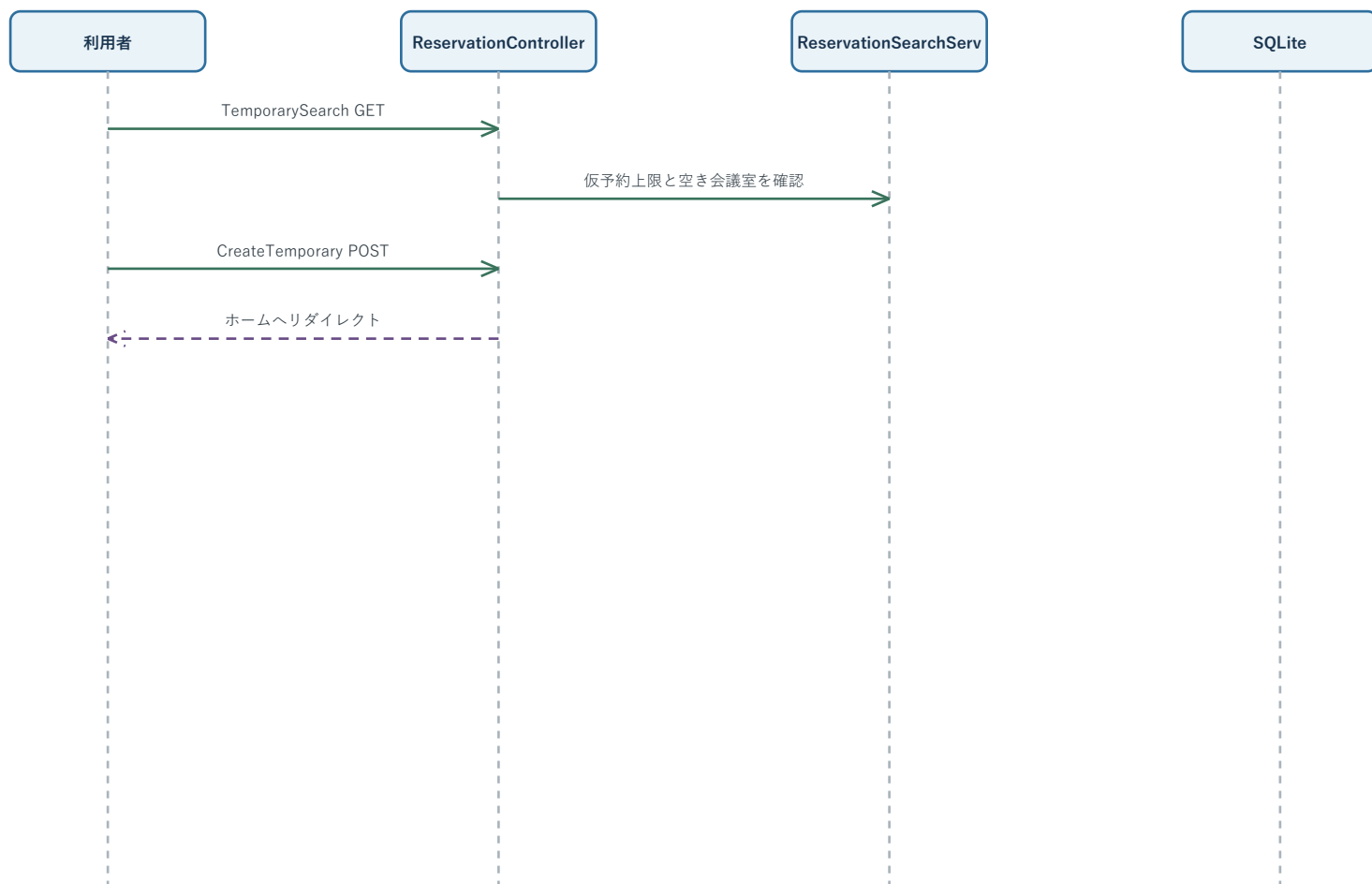
図：処理の呼び出し順

画面遷移は次のとおりです。

ReservationList/Index
→ Reservation/Search

- Reservation/Input
- Reservation/Confirm
- Reservation/Create
- ReservationList/Index

7. 新規仮予約フロー

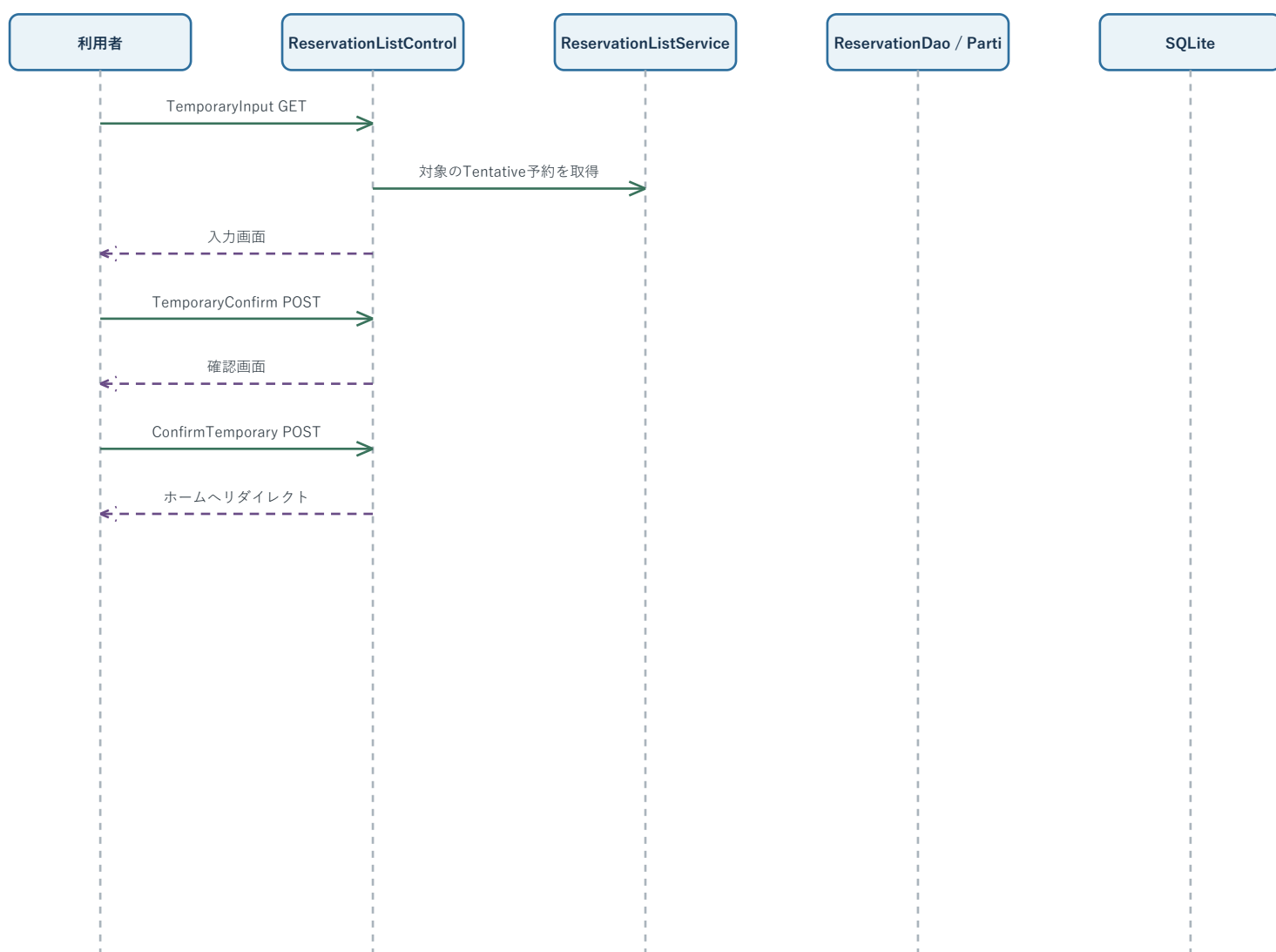


図：処理の呼び出し順

- ReservationList/Index
- Reservation/TemporarySearch
- Reservation/CreateTemporary
- ReservationList/Index

仮予約ではTitle、Remarks、TotalPriceをnullで保存できます。

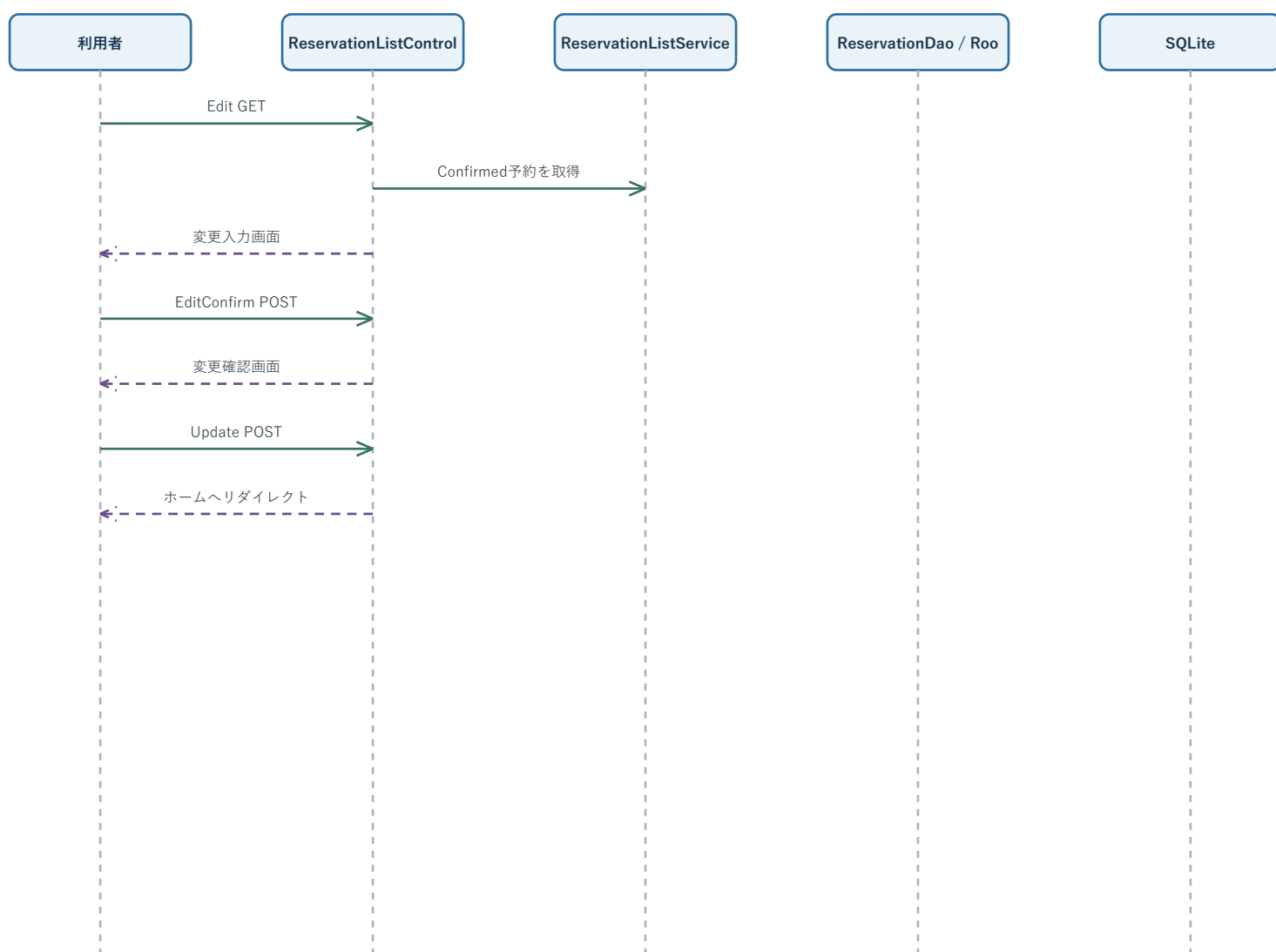
8. 仮予約から本予約へのフロー



図：処理の呼び出し順

既存のTentative予約を更新します。新しいReservationはINSERTしません。

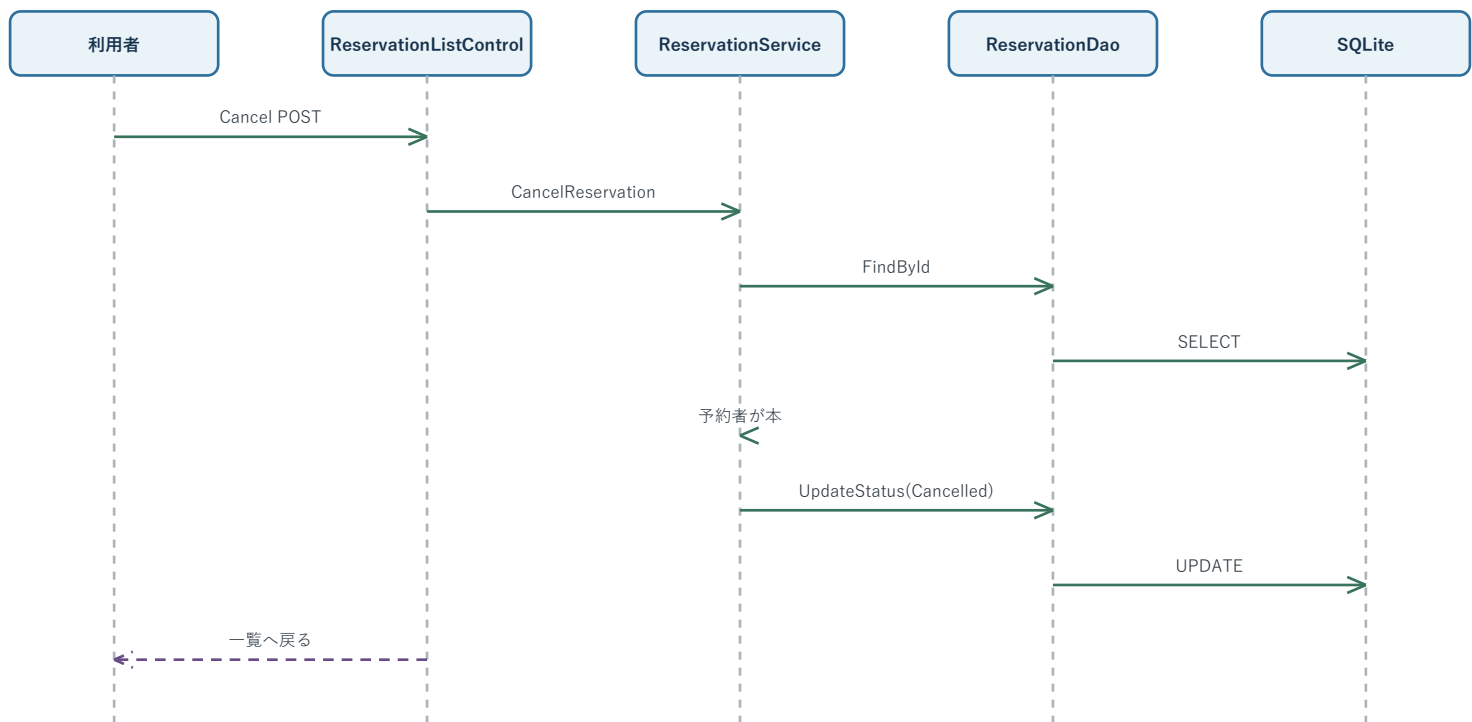
9. 予約変更フロー



図：処理の呼び出し順

変更時の空き判定では、変更対象の予約自身を除外します。

10. キャンセルフロー



図：処理の呼び出し順

キャンセルでは予約行を削除しません。StatusだけをCancelledへ変更し、Participantも残します。

11. タイムライン検索フロー

1. ホームで拠点、日付、人数を入力します。
2. **ReservationController.Search**がGET値を受け取ります。
3. 日付がない場合はModelStateへエラーを追加します。
4. **ReservationSearchService.GetReservationSchedule**を呼びます。
5. **FacilityDao.FindById**で拠点を確認します。
6. **RoomDao.FindByFacilityId**で会議室を取得します。
7. 会議室ごとに**ReservationDao.FindByRoomAndDate**を呼びます。
8. **ReservationScheduleViewModel**をViewへ渡します。
9. JavaScriptが09:00～18:00を15分ずつ、36スロットに分けます。
10. ConfirmedまたはTentativeと重なるスロットを押せない状態にします。
11. 開始スロットと終了スロットをクリックすると、連続範囲を選びます。
12. 既存予約をまたぐ範囲は拒否します。
13. 利用時間と料金プレビューを表示します。
14. 「次へ」でRoomId、RoomName、FacilityName、Date、StartTime、EndTimeを**Input**へGET送信します。

料金プレビューは画面表示専用です。正式なTotalPriceは送信せず、登録時にServiceで計算します。

12. Transactionの説明

Transactionは、複数のDB更新を「全部成功」または「全部取り消し」にする仕組みです。

現在の**ReservationService**では、次の4メソッドがそれぞれ自分の中でTransactionを管理します。

- **CreateReservation**
- **CreateTemporaryReservation**
- **ConfirmTemporaryReservation**
- **UpdateReservation**

基本形は次の順番です。

1. 入力値を確認する
2. SQLiteConnectionを開く
3. BeginTransactionでTransactionを開始する
4. tryへ入る
5. 同じConnectionとTransactionを渡してDAOを作る
6. 業務ルールを確認する
7. DAOを使ってDBを更新する
8. 全部成功したらCommitする
9. 条件不一致ならRollbackしてfalseを返す
10. 例外ならcatchでRollbackし、throwで同じ例外を呼び出し元へ返す

本予約作成ではReservationとParticipantを別テーブルへ登録します。同じTransactionを使うため、Participant登録に失敗した場合はReservation登録も取り消されます。

```

BeginTransaction
↓
reservationsへINSERT
↓
participantsへINSERT
↓
成功: Commit
失敗: Rollback

```

usingを抜けるとConnectionとTransactionは破棄されます。接続を閉じ忘れないために必要です。

13. 重複予約判定

ReservationDao.IsRoomAvailableは、指定された会議室と時間に重なる予約数を数えます。

重なるの条件は次のとおりです。

```

start_datetime < @requestedEnd
AND end_datetime > @requestedStart

```

既存予約が10:00～11:00の場合は次の結果になります。

09:00～10:00	空き。終了と開始が接するだけ
09:30～10:30	重複
10:30～11:30	重複
11:00～12:00	空き。終了と開始が接するだけ

Cancelledは空き判定から除外します。

予約変更時には`excludeReservationId`を指定し、変更対象の予約自身を除外します。自分自身を除外しないと、現在の予約が自分と重なっていると判断され、必ず変更に失敗してしまうためです。

検索時だけでなく登録直前にも判定します。検索画面を開いたあと、別の人が先に登録する可能性があるからです。

14. SQLiteParameter

利用者が入力した値をSQL文字列へ直接つなぐと、SQLインジェクションの危険があります。

このシステムでは、SQLと値を分けています。

```
const string sql = @"
SELECT room_id, room_name
FROM rooms
WHERE room_id = @roomId;";

using (SQLiteCommand command = CreateCommand(connection, transaction, sql))
{
    AddParameter(command, "@roomId", DbType.String, roomId);
}
```

`@roomId`が値を入れる場所です。`roomId`の内容はSQL命令ではなく、単なる値として扱われます。

`DaoBase.AddParameter`は、Parameter名、データ型、値を受け取ります。C#のnullはDB用の`DBNull.Value`へ変換します。

15. ViewData

画面の主なデータはModelまたはViewModelで渡します。画面表示を補助する小さなデータはViewDataで渡します。

現在の例です。

```
ViewData["Facilities"] = _reservationListService.GetFacilities();
ViewData["SelectedListType"] = listType;
```

Facilities	ホームの拠点プルダウン
SelectedListType	本予約・仮予約・実施済みのどのタブか
FacilityId	仮予約検索条件の再表示
Date	検索日付の再表示
StartTime	検索開始時刻の再表示
EndTime	検索終了時刻の再表示
Capacity	検索人数の再表示
Title	ページタイトル

ViewDataから値を取り出すときは、必要な型へ変換します。ModelStateの入力エラーはViewDataではなく、`Html.ValidationSummary`で表示します。

16. EntityとViewModelの違い

Entity

EntityはDBに保存されるデータを表します。

例として**Reservation**は、ReservationId、UserId、RoomId、開始・終了日時、Title、Status、TotalPriceなどをもちます。

ViewModel

ViewModelは特定の画面に必要なデータを表します。

例えばホーム画面では、予約情報だけでなくFacilityNameやRoomNameも表示します。これらをまとめるのが**ReservationDetailViewModel**です。

```
Reservation
+ Room
+ Facility
+ List<Participant>
  ↓ ReservationListServiceがまとめる
ReservationDetailViewModel
  ↓
Index.cshtml
```

なぜ分けるのか

- DBの形と画面の形は同じとは限らない
- Entityへ画面専用項目を増やさずに済む
- 入力検証を画面用モデルへ付けられる
- 1つの画面に複数テーブルの情報をまとめられる

ReservationInputViewModelの**RoomId**と**Title**には**Required**が付いています。POST時に必須入力不足すると、**ModelState.IsValid**がfalseになります。

17. DBテーブルとEntityの対応

users	User	user_id
facilities	Facility	facility_id
rooms	Room	room_id
reservations	Reservation	reservation_id
participants	Participant	reservation_id + participant_name + company_name

主な関連

```
Facility 1 —— * Room
Room 1 —— * Reservation
User 1 —— * Reservation
Reservation 1 —— * Participant
```

ParticipantはUserと関連付けません。ParticipantIdも持ちません。**Reservation**が**List<Participant>**を持ちます。

18. 初心者が理解しにくい箇所

1. Controllerのコンストラクタ

各Controllerには引数なしコンストラクタが1つだけあります。その中で**SQLiteConnectionFactory**と必要なServiceを直接作ります。Serviceを引数で受け取るテスト用コンストラクタや、Service生成専用のprivateメソッドは現在ありません。

2. nullable型

int?、**DateTime?**はnullを持てる型です。

- ・ 人数未指定を表す**capacity**
- ・ 仮予約で未確定の**TotalPrice**
- ・ GETで日付が送られなかった状態

3. using

Connection、Transaction、Command、DataReaderなどを使い終わったときに、自動で後片付けするための構文です。

4. DateTimeKind.Unspecified

DBにはJSTの時計表示をISO-8601 TEXTで保存します。タイムゾーン情報を文字列へ付けないため、DB用日時は**Unspecified**として扱います。

保存例：

```
2026-09-07T09:00:00.000
```

5. ASP.NET MVCのListモデルバインド

参加者入力欄は次の名前を使います。

```
Participants[0].Name  
Participants[0].CompanyName  
Participants[0].IsPrivate
```

この名前によって、POST時にASP.NET MVCが**List<Participant>**を作れます。**Participants.Index**のhidden項目も復元に使われます。

6. ControllerからViewを直接指定する箇所

ReservationListControllerは、仮予約確定と予約変更で次の実際のViewを直接指定します。

- ・ **TemporaryInput.cshtml**
- ・ **TemporaryConfirm.cshtml**
- ・ **Edit.cshtml**
- ・ **EditConfirm.cshtml**

以前存在した**Input.cshtml**、**ConfirmTemporary.cshtml**、**ConfirmUpdate.cshtml**という中継Viewは削除されています。入力エラー時も実際のView名へ直接戻ります。

7. checked

料金計算に**checked**を使用しています。計算結果が**int**の範囲を超えたとき、間違った金額をそのまま使わず例外にするためです。

19. セキュリティと安全性

ValidateAntiForgeryToken

DBを変更するPOST Actionには**ValidateAntiForgeryToken**が付いています。View側の**Html.AntiForgeryToken()**と組み合わせ、別サイトから勝手にPOSTされる攻撃を防ぎます。

所有者確認

仮予約確定、予約変更、キャンセルでは、予約のUserIdと現在のUserIdが同じかServiceで確認します。

DB制約

Initialize.sqlには、外部キー、CHECK、UNIQUEなどの制約があります。Serviceの確認に加えてDBでも不正なデータを防ぎます。

20. コードを読むおすすめ順

初めて読む場合は、次の順番がおすすめです。

1. **Models/Entities**を読み、扱うデータを知る
2. **Models/ViewModels**を読み、画面用データを知る
3. **Views/ReservationList/Index.cshtml**でホーム画面を確認する
4. Controllerメソッド一覧を見ながらControllerを読む
5. 新規予約なら**ReservationController**から追う
6. 既存予約操作なら**ReservationListController**から追う
7. Serviceメソッド一覧で業務ルールを確認する
8. Serviceが呼ぶDAOをDAOメソッド一覧で確認する
9. DAOのSQLと**Initialize.sql**の列を見比べる
10. 最後にTransaction、日時変換、タイムラインJavaScriptを読む

1つの機能だけを縦に追う方法もおすすめです。

Viewのform送信先
→ ControllerのAction
→ Serviceのメソッド
→ DAOのメソッド
→ SQL
→ DBテーブル

すべてを一度に覚える必要はありません。まず新規本予約かキャンセルのどちらか1つを選び、入口からDBまで順番に追うと理解しやすくなります。

付録A Controllerメソッド解説

この資料は、現在のControllerを上から順番に追うための補助資料です。

Controllerの仕事は、画面から値を受け取り、Serviceを呼び、次のViewまたはリダイレクト先を返すことです。SQL、料金計算、重複判定はControllerに書きません。

現在はログイン未実装のため、どちらのControllerも次の定数を使います。

```
private const string CurrentUserId = "U001";
```

将来は、この定数をSessionなどからログインユーザーIDを取得する処理へ置き換えます。

ReservationController

新しい本予約・仮予約を作る流れを担当します。

ReservationController()

- ・ 引数：なし
- ・ 戻り値：なし（コンストラクタ）
- ・ 呼び出し元：ASP.NET MVC
- ・ 次に作るもの：**SQLiteConnectionFactory**、**ReservationSearchService**、**ReservationService**
- ・ 処理：2つのServiceを直接生成します。Serviceを引数で受け取る別コンストラクタや、Service生成専用メソッドはありません。

Search(string facilityId, DateTime? date, int? capacity)

- ・ HTTP：GET
- ・ 引数：拠点ID、日付、任意の人数
- ・ 戻り値：**Search.cshtml**と**ReservationScheduleViewModel**
- ・ 呼び出し元：ホームの新規本予約検索フォーム
- ・ 次に呼ぶもの：**ReservationSearchService.GetReservationSchedule**
- ・ 処理順：日付があるか確認し、Serviceから拠点内の会議室と当日の予約を取得し、タイムライン画面へ渡します。取得できない場合は**ModelState**へエラーを追加します。

TemporarySearch(string facilityId, DateTime? date, TimeSpan? startTime, TimeSpan? endTime, int? capacity)

```
TemporarySearch(  
    string facilityId,  
    DateTime? date,  
    TimeSpan? startTime,  
    TimeSpan? endTime,  
    int? capacity)
```

- ・ HTTP：GET
- ・ 戻り値：**TemporarySearch.cshtml**と**List<Room>**
- ・ 呼び出し元：ホームの新規仮予約検索フォーム

- ・ 次に呼ぶもの：**CanCreateTemporaryReservation**、**SearchAvailableRoomsForTemporary**
- ・ 処理順：検索条件を**ViewData**へ保存し、必須日時と仮予約3件上限を確認してから、空いている会議室を取得します。

Input(string roomId, string roomName, string facilityName, DateTime date, TimeSpan startTime, TimeSpan endTime)

```
Input(  
    string roomId,  
    string roomName,  
    string facilityName,  
    DateTime date,  
    TimeSpan startTime,  
    TimeSpan endTime)
```

- ・ HTTP：GET
- ・ 戻り値：**Input.cshtml**と**ReservationInputViewModel**
- ・ 呼び出し元：本予約タイムラインの「次へ」
- ・ 次に呼ぶもの：Serviceは呼ばない
- ・ 処理：タイムラインから受け取った値を入力用ViewModelへ詰め替えます。この時点ではDBへ登録しません。

Confirm(ReservationInputViewModel model)

- ・ HTTP：POST
- ・ セキュリティ：**ValidateAntiForgeryToken**
- ・ 戻り値：正常時は**Confirm.cshtml**、入力エラー時は**Input.cshtml**
- ・ 呼び出し元：新規本予約入力画面
- ・ 次に呼ぶもの：Serviceは呼ばない
- ・ 処理：**ModelState.IsValid**を確認し、入力内容を確認画面へ渡します。DB更新はありません。

Create(ReservationInputViewModel model)

- ・ HTTP：POST
- ・ セキュリティ：**ValidateAntiForgeryToken**
- ・ 戻り値：成功時は**ReservationList/Index**へリダイレクト、失敗時は**Confirm.cshtml**
- ・ 呼び出し元：新規本予約確認画面
- ・ 次に呼ぶもの：**ReservationService.CreateReservation**
- ・ 処理：入力検証後にServiceへ登録を依頼します。正式料金と重複判定はServiceが行います。

CreateTemporary(string roomId, DateTime date, TimeSpan startTime, TimeSpan endTime)

```
CreateTemporary(  
    string roomId,  
    DateTime date,  
    TimeSpan startTime,  
    TimeSpan endTime)
```

- HTTP : POST
- セキュリティ : **ValidateAntiForgeryToken**
- 戻り値 : 成功時は**ReservationList/Index**へリダイレクト、失敗時は**TemporarySearch.cshtml**
- 呼び出し元 : 仮予約検索結果
- 次に呼ぶもの : **ReservationService.CreateTemporaryReservation**
- 処理 : 選択された会議室と日時をServiceへ渡します。上限、営業時間、空き状況はServiceが登録直前に再確認します。

ReservationListController

ホーム表示と、すでに存在する予約の操作を担当します。

ReservationListController()

- 引数 : なし
- 戻り値 : なし (コンストラクタ)
- 呼び出し元 : ASP.NET MVC
- 次に作るもの : **SQLiteConnectionFactory**、**ReservationListService**、**ReservationService**
- 処理 : 2つのServiceを直接生成します。Serviceを受け取る別コンストラクタはありません。

Index(string listType = "confirmed")

- HTTP : GET
- 引数 : **confirmed**、**temporary**、**completed**のいずれか
- 戻り値 : **Index.cshtml**と**List<ReservationDetailViewModel>**
- 呼び出し元 : 既定ルート、各処理後のリダイレクト、一覧タブ
- 次に呼ぶもの : **GetReservationsForList**、**ReservationListService.GetFacilities**
- 処理順 : 不明な一覧種別を**confirmed**へ直し、一覧を取得します。拠点一覧を**ViewData["Facilities"]**、選択中タブを**ViewData["SelectedListType"]**へ設定します。

Cancel(int reservationId, string listType = "confirmed")

- HTTP : POST
- セキュリティ : **ValidateAntiForgeryToken**
- 戻り値 : 成功時は同じ一覧種別へリダイレクト、失敗時は**Index.cshtml**
- 呼び出し元 : 本予約・仮予約一覧のキャンセルフォーム
- 次に呼ぶもの : **ReservationService.CancelReservation**
- 処理 : 一覧種別を確認してServiceへキャンセルを依頼します。失敗時はIndexに必要な予約一覧、拠点一覧、選択中タブを設定し直します。

GetReservationsForList(string listType)

- ・ 種類：private補助メソッド
- ・ 戻り値：**List<ReservationDetailViewModel>**
- ・ 呼び出し元：**Index**、**Cancel**のエラー処理
- ・ 次に呼ぶもの：一覧種別に応じて次のいずれか

temporary	GetReservationList(U001, Tentative)
completed	GetCompletedReservations(U001)
その他	GetCurrentConfirmedReservations(U001)

TemporaryInput(int reservationId)

- ・ HTTP：GET
- ・ 戻り値：**TemporaryInput.cshtml**、見つからなければ404
- ・ 呼び出し元：仮予約一覧の「予約確定」
- ・ 次に呼ぶもの：**FindReservationDetail**、**ToInputViewModel**
- ・ 処理：U001のTentative予約を探し、入力用ViewModelへ変換して実際の入力Viewを直接表示します。

TemporaryConfirm(int reservationId, ReservationInputViewModel model)

- ・ HTTP：POST
- ・ セキュリティ：**ValidateAntiForgeryToken**
- ・ 戻り値：正常時は**TemporaryConfirm.cshtml**、入力エラー時は**TemporaryInput.cshtml**
- ・ 呼び出し元：仮予約確定入力画面
- ・ 次に呼ぶもの：Serviceは呼ばない
- ・ 処理：入力検証後、予約IDをViewModelへ設定して確認画面へ渡します。

ConfirmTemporary(int reservationId, ReservationInputViewModel model)

- ・ HTTP：POST
- ・ セキュリティ：**ValidateAntiForgeryToken**
- ・ 戻り値：成功時はIndexへリダイレクト、失敗時は**TemporaryConfirm.cshtml**
- ・ 呼び出し元：仮予約確定確認画面
- ・ 次に呼ぶもの：**ReservationService.ConfirmTemporaryReservation**
- ・ 処理：既存のTentative予約をConfirmedへ更新するようServiceへ依頼します。新しい予約は作りません。

Edit(int reservationId)

- ・ HTTP：GET
- ・ 戻り値：**Edit.cshtml**、見つからなければ404
- ・ 呼び出し元：本予約一覧の「変更」

- ・ 次と呼ぶもの：**FindReservationDetail**、**ToInputViewModel**
- ・ 処理：U001のConfirmed予約を探し、現在値を入力用ViewModelへ移して変更画面を直接表示します。

EditConfirm(ReservationInputViewModel model)

- ・ HTTP：POST
- ・ セキュリティ：**ValidateAntiForgeryToken**
- ・ 戻り値：正常時は**EditConfirm.cshtml**、入力エラー時は**Edit.cshtml**
- ・ 呼び出し元：予約変更入力画面
- ・ 次と呼ぶもの：Serviceは呼ばない
- ・ 処理：入力内容を検証し、変更確認画面へ渡します。

Update(ReservationInputViewModel model)

- ・ HTTP：POST
- ・ セキュリティ：**ValidateAntiForgeryToken**
- ・ 戻り値：成功時はIndexへリダイレクト、失敗時は**EditConfirm.cshtml**
- ・ 呼び出し元：予約変更確認画面
- ・ 次と呼ぶもの：**ReservationService.UpdateReservation**
- ・ 処理：Serviceへ既存Confirmed予約の更新を依頼します。

FindReservationDetail(int reservationId, ReservationStatus status)

- ・ 種類：private補助メソッド
- ・ 戻り値：一致した**ReservationDetailViewModel**。なければnull
- ・ 呼び出し元：**TemporaryInput**、**Edit**
- ・ 次と呼ぶもの：**ReservationListService.GetReservationList**
- ・ 処理：U001と状態で一覧を取得し、その中から予約IDが一致する1件を探します。

ToInputViewModel(ReservationDetailViewModel detail)

- ・ 種類：private static補助メソッド
- ・ 戻り値：**ReservationInputViewModel**
- ・ 呼び出し元：**TemporaryInput**、**Edit**
- ・ 処理：一覧・詳細用の値を入力画面用の形へ詰め替えます。**RoomId**は変更処理に必要な内部値です。

中継Viewについて

Controllerは現在、**TemporaryInput.cshtml**、**TemporaryConfirm.cshtml**、**Edit.cshtml**、**EditConfirm.cshtml**を直接指定します。以前の中継Viewである**Input.cshtml**、**ConfirmTemporary.cshtml**、**ConfirmUpdate.cshtml**は**Views/ReservationList**には存在しません。

付録B Serviceメソッド解説

ServiceはControllerとDAOの間にあり、予約の業務ルールを判断します。Service内にSQLはありません。

ReservationSearchService

ReservationSearchService(SQLiteConnectionFactory connectionFactory)

- ・ 引数：SQLite接続を作るFactory
- ・ 戻り値：なし（コンストラクタ）
- ・ 呼び出し元：**ReservationController()**
- ・ 処理：引数がnullでないことを確認し、**FacilityDao**、**RoomDao**、**ReservationDao**を作ります。

CanCreateTemporaryReservation(string userId)

- ・ 引数：ユーザーID
- ・ 戻り値：仮予約が3件未満ならtrue、それ以外はfalse
- ・ 呼び出し元：**ReservationController.TemporarySearch**、**SearchAvailableRoomsForTemporary**
- ・ 次に呼ぶもの：**ReservationDao.CountTemporaryReservations**
- ・ 処理：空のユーザーIDを拒否し、現在のTentative件数を数えます。

GetReservationSchedule(string facilityId, DateTime date, int? capacity)

- ・ 戻り値：**ReservationScheduleViewModel**。条件不正や拠点なしならnull
- ・ 呼び出し元：**ReservationController.Search**
- ・ 次に呼ぶもの：**FacilityDao.FindById**、**RoomDao.FindByFacilityId**、**ReservationDao.FindByRoomAndDate**
- ・ 処理順：
 1. 拠点IDと人数を確認します。
 2. 拠点を取得します。
 3. 画面全体用の**ReservationScheduleViewModel**を作ります。
 4. 条件に合う会議室を取得します。
 5. **foreach**で各会議室の当日予約を取得します。
 6. 会議室1行分の**RoomScheduleViewModel**を追加して返します。

SearchAvailableRoomsForTemporary(string facilityId, DateTime date, TimeSpan startTime, TimeSpan endTime, int? capacity, string userId)

```
SearchAvailableRoomsForTemporary(
    string facilityId,
    DateTime date,
    TimeSpan startTime,
    TimeSpan endTime,
    int? capacity,
    string userId)
```

- ・ 戻り値：指定時間に空いている **List<Room>**
- ・ 呼び出し元：**ReservationController.TemporarySearch**
- ・ 次に呼ぶもの：**CanCreateTemporaryReservation**、**RoomDao.FindByFacilityId**、**ReservationDao.IsRoomAvailable**
- ・ 処理順：検索条件、3件上限、営業時間、15分単位を確認します。会議室を1件ずつ空き判定し、空いているものだけを結果へ追加します。

IsValidReservationTime(TimeSpan startTime, TimeSpan endTime)

- ・ 種類：private static補助メソッド
- ・ 戻り値：有効ならtrue
- ・ 確認内容：09:00以降、18:00以前、開始より終了が後、開始・終了が15分単位

CombineDateAndTime(DateTime date, TimeSpan time)

- ・ 種類：private static補助メソッド
- ・ 戻り値：日付と時刻を合わせた**DateKind.Unspecified**の日時
- ・ 理由：DBにはJSTの時計表示をタイムゾーンなしのISO-8601 TEXTで保存するためです。

ReservationService

DBを変更する予約処理を担当します。次の4メソッドは、各メソッド内で同じConnectionとTransactionをDAOへ渡します。

- ・ **CreateReservation**
- ・ **CreateTemporaryReservation**
- ・ **ConfirmTemporaryReservation**
- ・ **UpdateReservation**

Transactionは共通メソッドへ処理を渡さず、各メソッドの中に**using → BeginTransaction → try → DAO作成 → Commit / Rollback**の順で書かれています。

ReservationService(SQLiteConnectionFactory connectionFactory)

- ・ 引数：SQLite接続を作るFactory
- ・ 呼び出し元：2つのControllerの引数なしコンストラクタ
- ・ 処理：nullを拒否し、後でConnectionを開くためにFactoryを保持します。

CreateReservation(ReservationInputViewModel model, string userId)

- ・ 戻り値：登録成功ならtrue、条件不正ならfalse。DB例外は再スロー
- ・ 呼び出し元：**ReservationController.Create**
- ・ 次と呼ぶDAO：**RoomDao**、**ReservationDao**、**ParticipantDao**

処理順は次のとおりです。

1. **TryGetReservationRange**でViewModel、RoomId、営業時間、15分単位を確認します。
2. userIdとTitleが空でないことを確認します。
3. Connectionを開きます。
4. **BeginTransaction()**でTransactionを開始します。
5. **try**の中で、同じConnectionとTransactionを渡して3つのDAOを作ります。
6. **IsRoomAvailable**で登録直前の空き状況を再確認します。
7. **RoomDao.FindById**でRoomとHourlyRateを取得します。
8. **CalculateTotalPrice**で正式料金を計算します。
9. 参加者リストをコピーし、Confirmedの**Reservation**を作ります。
10. **ReservationDao.Insert**で予約を登録します。
11. **ParticipantDao.InsertAll**で参加者を登録します。
12. 採番されたReservationIdと正式料金をViewModelへ戻します。
13. **Commit()**してtrueを返します。
14. 条件不一致では**Rollback()**してfalseを返します。
15. 例外時は**catch**で**Rollback()**し、**throw**で例外をそのまま上へ返します。

CreateTemporaryReservation(string roomId, DateTime date, TimeSpan startTime, TimeSpan endTime, string userId)

```
CreateTemporaryReservation(  
    string roomId,  
    DateTime date,  
    TimeSpan startTime,  
    TimeSpan endTime,  
    string userId)
```

- ・ 戻り値：登録成功ならtrue、それ以外はfalse
- ・ 呼び出し元：**ReservationController.CreateTemporary**
- ・ 次と呼ぶDAO：**RoomDao**、**ReservationDao**

処理順：

1. roomId、userId、営業時間、15分単位を確認します。
2. 日付と時刻を開始・終了DateTimeへ合わせます。
3. ConnectionとTransactionを開始します。

4. 同じConnectionとTransactionでDAOを作ります。
5. Tentativeが3件未満か登録直前に再確認します。
6. Roomが存在するか確認します。
7. 指定時間が空いているか再確認します。
8. Title、Remarks、TotalPriceがnullのTentative予約を作ります。
9. **ReservationDao.Insert**で登録します。
10. 成功時はCommitしてtrue、条件不一致はRollbackしてfalse、例外時はRollback後に再スローします。

ConfirmTemporaryReservation(int reservationId, ReservationInputViewModel model, string userId)

- ・ 戻り値：本予約への更新成功ならtrue、それ以外はfalse
- ・ 呼び出し元：**ReservationListController.ConfirmTemporary**
- ・ 次に呼ぶDAO：**RoomDao**、**ReservationDao**、**ParticipantDao**

処理順：

1. model、userId、Titleを確認します。
2. ConnectionとTransactionを開始します。
3. 同じConnectionとTransactionでDAOを作ります。
4. **FindById**で既存予約を取得します。
5. 予約者がuserIdと同じか確認します。
6. StatusがTentativeか確認します。
7. 開始日と終了日が同じか、営業時間内か、15分単位か確認します。
8. 自分の予約IDを除外して空き状況を再確認します。
9. Roomを取得し、正式料金を計算します。
10. Title、Remarks、Status、TotalPrice、Participantsを更新します。
11. **ReservationDao.Update**で既存予約を更新します。
12. 古い参加者を削除し、新しい参加者を登録します。
13. 成功時はCommitします。途中失敗と例外時の扱いは本予約作成と同じです。

新しいReservationはINSERTしません。既存のTentative行をConfirmedへ変えます。

UpdateReservation(ReservationInputViewModel model, string userId)

- ・ 戻り値：変更成功ならtrue、それ以外はfalse
- ・ 呼び出し元：**ReservationListController.Update**
- ・ 次に呼ぶDAO：**RoomDao**、**ReservationDao**、**ParticipantDao**

処理順：

1. 入力日時、ReservationId、userId、Titleを確認します。
2. ConnectionとTransactionを開始します。
3. 既存予約を取得します。
4. 予約者が本人か確認します。
5. StatusがConfirmedか確認します。
6. 自分のReservationIdを除外して、新しいRoomと日時の空さを確認します。
7. RoomとHourlyRateを取得します。
8. Room、日時、Title、Remarks、正式料金、Participantsを新しい値へ変えます。
9. 予約を更新します。
10. 参加者を削除してから登録し直します。
11. 成功時はCommit、条件不一致はRollback、例外時はRollback後に再スローします。

CancelReservation(int reservationId, string userId)

- ・ 戻り値：キャンセル成功ならtrue、それ以外はfalse
- ・ 呼び出し元：**ReservationListController.Cancel**
- ・ 次と呼ぶもの：**ReservationDao.FindById**、**ReservationDao.UpdateStatus**
- ・ 処理：userIdが空でないこと、予約が存在すること、予約者本人であることを確認し、StatusをCancelledへ変更します。Reservation行とParticipantは削除しません。このメソッドは複数テーブル更新を行わないため、明示的なTransactionを開始していません。

TryGetReservationRange(ReservationInputViewModel model, out DateTime startDateTime, out DateTime endDateTime)

- ・ 種類：private static補助メソッド
- ・ 戻り値：日時を作ればtrue
- ・ 処理：modelとRoomIdを確認し、**IsValidReservationTime**を通過したら日付と開始・終了時刻をDateTimeへまとめます。結果は**out**引数へ入れます。

IsValidReservationTime(TimeSpan startTime, TimeSpan endTime)

- ・ 種類：private static補助メソッド
- ・ 戻り値：営業時間内かつ15分単位ならtrue

CombineDateAndTime(DateTime date, TimeSpan time)

- ・ 種類：private static補助メソッド
- ・ 戻り値：日付と時刻を合わせた**DateTimeKind.Unspecified**の日時

CalculateTotalPrice(int hourlyRate, DateTime startDateTime, DateTime endDateTime)

- ・ 種類：private static補助メソッド
- ・ 戻り値：整数円の正式料金
- ・ 計算：**HourlyRate / 4 × 15分スロット数**
- ・ 補足：**checked**により、計算結果がintの範囲を超えた場合は例外になります。

CopyParticipants(IEnumerable<Participant> participants)

- ・ 種類：private static補助メソッド
- ・ 戻り値：新しい**List<Participant>**
- ・ 処理：nullなら空リスト、それ以外は受け取った参加者を新しいリストへコピーします。

ReservationListService

ReservationListService(SQLiteConnectionFactory connectionFactory)

- ・ 呼び出し元：**ReservationListController()**
- ・ 処理：Factoryがnullでないことを確認し、Facility、Room、Reservation、ParticipantのDAOを作ります。

GetReservationList(string userId, ReservationStatus status)

- ・ 戻り値：**List<ReservationDetailViewModel>**
- ・ 呼び出し元：一覧Controllerの補助メソッドと詳細検索
- ・ 次に呼ぶもの：**FindByUserAndStatus**、**RoomDao.FindById**、**FacilityDao.FindById**、**ParticipantDao.FindByReservationId**
- ・ 処理：ユーザーと状態で予約を取得し、会議室、拠点、参加者を1件ずつ取得して画面用ViewModelへまとめます。RoomまたはFacilityが見つからない予約は結果へ追加しません。

GetFacilities()

- ・ 戻り値：**List<Facility>**
- ・ 呼び出し元：**ReservationListController.Index**とCancel失敗時
- ・ 次に呼ぶもの：**FacilityDao.FindAll**
- ・ 処理：ホーム画面の拠点プルダウン用一覧を返します。

GetCurrentConfirmedReservations(string userId)

- ・ 戻り値：これから実施する本予約の一覧
- ・ 呼び出し元：**ReservationListController.GetReservationsForList**
- ・ 処理：Confirmedを取得し、**EndTime >= DateTime.Now**の予約だけを返します。

GetCompletedReservations(string userId)

- ・ 戻り値：実施済み予約の一覧

- ・ 呼び出し元：**ReservationListController.GetReservationsForList**
- ・ 処理：Confirmedを取得し、**EndTime < DateTime.Now**の予約だけを返します。実施済み専用のReservationStatusはありません。

付録C DAOメソッド解説

DAOはSQLiteへSQLを実行し、結果をEntityへ変換します。画面遷移、料金、仮予約上限などの業務判断は行いません。

すべての入力値は**SQLiteParameter**でSQLへ渡します。

各DAOの2種類のコンストラクタ

次のDAOには、同じ形のコンストラクタが2種類ずつあります。

- **FacilityDao(SQLiteConnectionFactory connectionFactory) / FacilityDao(SQLiteConnection connection, SQLiteTransaction transaction)**
- **RoomDao(SQLiteConnectionFactory connectionFactory) / RoomDao(SQLiteConnection connection, SQLiteTransaction transaction)**
- **ReservationDao(SQLiteConnectionFactory connectionFactory) / ReservationDao(SQLiteConnection connection, SQLiteTransaction transaction)**
- **ParticipantDao(SQLiteConnectionFactory connectionFactory) / ParticipantDao(SQLiteConnection connection, SQLiteTransaction transaction)**

Factory版はDAOが単独で接続を開く場合に使います。ConnectionとTransactionを受け取る版は、**ReservationService**が複数DAOを同じTransactionへ参加させる場合に使います。

SQLiteConnectionFactory

SQLiteConnectionFactory()

- 引数：なし
- 呼び出し元：Controllerの引数なしコンストラクタ
- 処理：**Web.config**の**ReservationDatabase**接続文字列を直接読みます。設定がない、または空なら**ConfigurationErrorsException**を投げます。

SQLiteConnectionFactory(string connectionString)

- 引数：接続文字列
- 用途：任意のSQLite DBへ接続したい場合
- 処理：空の接続文字列を拒否して保持します。

OpenConnection()

- 戻り値：開かれた**SQLiteConnection**
- 呼び出し元：**ReservationService**、**DaoBase.Execute**
- 処理：Connectionを開き、**PRAGMA foreign_keys = ON;**を実行してから返します。SQLiteでは接続ごとに外部キー制約を有効にする必要があります。

GetConnectionString()という補助メソッドは現在ありません。既定コンストラクタが設定取得を直接行います。

DaoBase

各DAOが継承する共通クラスです。

DaoBase(SQLiteConnectionFactory connectionFactory)

- ・ 用途：DAOが単独でDB操作するとき
- ・ 処理：Factoryを保持し、必要になった時点でConnectionを開けるようにします。

DaoBase(SQLiteConnection connection, SQLiteTransaction transaction)

- ・ 用途：Serviceが複数DAOで同じTransactionを使うとき
- ・ 処理：Connection、Transaction、接続状態を確認して共有用フィールドへ保持します。

Execute<T>(Func<SQLiteConnection, SQLiteTransaction, T> operation)

- ・ 戻り値：渡された処理の結果T
- ・ 呼び出し元：各DAOメソッド
- ・ 処理：共有Connectionがあればそれを使います。なければFactoryからConnectionを開きます。
- ・ 補足：DAOには現在もFuncとラムダ式によるこの共通化があります。ReservationServiceのTransaction処理とは別の仕組みです。

CreateCommand(SQLiteConnection connection, SQLiteTransaction transaction, string commandText)

- ・ 戻り値：SQLとTransactionが設定されたSQLiteCommand
- ・ 処理：ConnectionからCommandを作り、SQL文字列とTransactionを設定します。

AddParameter(SQLiteCommand command, string name, DbType type, object value)

- ・ 処理：Parameter名、DbType、値をCommandへ追加します。C#のnullはDBNull.Valueへ変換します。

ToDatabaseDateTime(DateTime value)

- ・ 戻り値：JSTローカル時刻のISO-8601文字列
- ・ 保存形式：yyyy-MM-dd'T'HH:mm:ss.fff
- ・ 処理：UTCまたはPCローカル時刻なら東京時刻へ変換し、Unspecifiedなら入力された時計表示をそのまま使います。

FromDatabaseDateTime(string value)

- ・ 戻り値：DateTimeKind.UnspecifiedのDateTime
- ・ 処理：ミリ秒あり・なしのISO-8601文字列を読みます。対応外の形式ならFormatExceptionを投げます。

FacilityDao

2つのコンストラクタがあります。Factory版は通常の単独読み取り用、ConnectionとTransaction版はServiceから共有する場合に使います。

FindAll()

- ・ 戻り値：全拠点の**List<Facility>**
- ・ 呼び出し元：**ReservationListService.GetFacilities**
- ・ SQL：facilitiesを**facility_id**順でSELECTします。

FindById(string facilityId)

- ・ 戻り値：該当する**Facility**、なければnull
- ・ 呼び出し元：検索Service、一覧Service
- ・ SQL：**facility_id = @facilityId**で1件取得します。

MapFacility(SQLiteDataReader reader)

- ・ 種類：private static補助メソッド
- ・ 処理：DataReaderの1行を**Facility**へ変換します。

RoomDao

FindByFacilityId(string facilityId, int? capacity)

- ・ 戻り値：条件に合う**List<Room>**
- ・ 呼び出し元：**ReservationSearchService**
- ・ SQL：拠点を必須条件にし、capacityがある場合だけ定員以上の会議室を取得します。

FindById(string roomId)

- ・ 戻り値：該当する**Room**、なければnull
- ・ 呼び出し元：予約Service、一覧Service
- ・ SQL：**room_id = @roomId**で1件取得します。

MapRoom(SQLiteDataReader reader)

- ・ 種類：private static補助メソッド
- ・ 処理：DataReaderの1行を**Room**へ変換します。

ReservationDao

FindById(int reservationId)

- ・ 戻り値：該当する**Reservation**、なければnull
- ・ 呼び出し元：予約確定、変更、キャンセル
- ・ 補足：Participantsは取得しません。参加者は**ParticipantDao**の責務です。

FindByRoomAndDate(string roomId, DateTime date)

- ・ 戻り値：会議室の指定日に重なる予約一覧
- ・ 呼び出し元：本予約タイムライン検索
- ・ SQL条件：日付範囲と重なる予約を取得し、Cancelledを除外します。

FindByUserAndStatus(string userId, ReservationStatus status)

- ・ 戻り値：ユーザーと状態が一致する予約一覧
- ・ 呼び出し元：**ReservationListService.GetReservationList**

CountTemporaryReservations(string userId)

- ・ 戻り値：ユーザーが持つTentative予約の件数
- ・ 呼び出し元：検索Service、予約Service
- ・ 用途：仮予約3件上限の確認

IsRoomAvailable(string roomId, DateTime start, DateTime end, int? excludeReservationId = null)

- ・ 戻り値：重なる予約が0件ならtrue
- ・ 呼び出し元：検索Service、予約Service
- ・ 重複条件：

```
start_datetime < @requestedEnd
AND end_datetime > @requestedStart
AND status <> @cancelledStatus
```

excludeReservationIdが指定された場合だけ、変更対象自身を除外する条件を追加します。

```
AND reservation_id <> @excludeReservationId
```

Insert(Reservation reservation)

- ・ 戻り値：SQLiteが自動採番したReservationId
- ・ 呼び出し元：本予約作成、仮予約作成
- ・ SQL：reservationsへINSERTし、**last_insert_rowid()**を取得します。

Update(Reservation reservation)

- ・ 戻り値：1件以上更新できればtrue
- ・ 呼び出し元：仮予約確定、予約変更
- ・ SQL：ReservationIdが一致する予約の各項目をUPDATEします。

UpdateStatus(int reservationId, ReservationStatus status)

- ・ 戻り値：更新できればtrue
- ・ 呼び出し元：**ReservationService.CancelReservation**
- ・ SQL：Statusだけを更新します。

AddReservationParameters(SQLiteCommand command, Reservation reservation)

- ・ 種類：private static補助メソッド
- ・ 処理：INSERTとUPDATEで共通するReservationのParameterをCommandへ追加します。

MapReservation(SQLiteDataReader reader)

- ・ 種類：private static補助メソッド
- ・ 処理：DataReaderの1行を**Reservation**へ変換します。DBの日時TEXTは**FromDatabaseDateTime**でDateTimeへ戻します。

ParticipantDao

FindByReservationId(int reservationId)

- ・ 戻り値：指定予約の**List<Participant>**
- ・ 呼び出し元：**ReservationListService.GetReservationList**
- ・ SQL：reservation_idが一致する参加者を取得します。

InsertAll(int reservationId, List<Participant> participants)

- ・ 戻り値：登録完了ならtrue
- ・ 呼び出し元：本予約作成、仮予約確定、予約変更
- ・ 処理：参加者を1件ずつパラメータ化したINSERTで登録します。
- ・ Transaction：ServiceからTransactionを渡された場合はそれを使います。単独で呼ばれた場合はParticipantDao自身がローカルTransactionを開始します。

DeleteByReservationId(int reservationId)

- ・ 戻り値：削除件数。0件でもエラーではありません。
- ・ 呼び出し元：仮予約確定、予約変更
- ・ 処理：新しい参加者へ入れ替える前に、指定予約の既存参加者を削除します。

InsertParticipants(SQLiteConnection connection, SQLiteTransaction transaction, int reservationId, IEnumerable<Participant> participants)

- ・ 種類：private static補助メソッド

- ・ 処理：**foreach**で参加者を登録します。リスト内にnullがあれば例外にします。CompanyNameがnullなら空文字を保存します。

UserDaoについて

UserDaoは現在のプロジェクトには存在しません。ログイン機能とユーザー検索機能が未実装で、現在の機能から使われていなかったため削除されています。

ただし、将来のログイン機能に備えて、**User** Entity、**UserRole** enum、**users**テーブルは残っています。